

Cours de Programmation Orientée Objet en python

Licence 2 MPI & SML

Année académique: 2024-2025

Pr Amadou Dahirou GUEYE

Plan du cours

Chapitre 0: Rappel sur les bases en python

- Bases, Boucles et structures de données, fonctions

Chapitre 1 : Introduction à la POO

- Notion de programmation procédurale vs orientée objet
- Concepts fondamentaux : objets, classes, attributs, méthodes
- Avantages de la POO
- Premier exemple de classe et d'objet en Python

Chapitre 2 : Les classes et objets en détail

- Définition et instanciation de classes
- Constructeur `__init__` et `self`
- Manipulation des attributs et des méthodes

Plan du cours

Chapitre 3 : Encapsulation et Modificateurs d'accès

- Attributs publics, protégés (`_`) et privés (`__`)
- Getters et Setters
- Propriétés avec `@property`

Chapitre 4 : L'Héritage et la Réutilisation du Code

- Principe de l'héritage
- Héritage simple et multiple
- Surcharge et redéfinition de méthodes (`super()`)

Chapitre 5 : Polymorphisme et Abstraction

- Méthodes polymorphes
- Classes et méthodes abstraites (`ABC` et `@abstractmethod`)
- Interfaces en Python

Plan du cours

Chapitre 6 : Gestion des Exceptions et Bonnes Pratiques

- Introduction aux exceptions en Python
- `try`, `except`, `finally`, `raise`
- Création d'exceptions personnalisées

Chapitre 7 : Programmation Orientée Objet Avancée

- Méthodes et attributs de classe (`@classmethod`, `@staticmethod`)
- Surcharge des opérateurs (`__add__`, `__eq__`, etc.)
- Design Patterns courants (Singleton, Factory, etc.)

Chapitre 8 : Manipulation des Fichiers et Sérialisation

- Lecture et écriture de fichiers (`open()`, `with`)
- Sérialisation avec `pickle` et `json`

Plan du cours

Chapitre 9 : Introduction aux Bibliothèques et Frameworks POO

- Introduction à `Tkinter`, `PyQt` pour les interfaces graphiques
- Notions de POO dans Django ou Flask

Chapitre 10 : Projet Final

- Application complète utilisant la POO
- Exemple : gestion d'un système de réservation, application de gestion de stocks, etc.

Chapitre 0: Rappel sur les bases en python

Présentation des concepts fondamentaux de
Python

INTRODUCTION

Python est un **langage de programmation interprété**, ce qui signifie que son exécution se fait ligne par ligne, sans nécessiter de compilation préalable. Il est apprécié pour sa **syntaxe claire et lisible**, ce qui en fait un excellent choix pour les débutants comme pour les experts.

INTRODUCTION

Pourquoi apprendre Python ?

Python est utilisé dans de nombreux domaines, notamment :

- Intelligence artificielle (IA) et apprentissage automatique : avec des bibliothèques comme TensorFlow, PyTorch et scikit-learn.
- Développement web : avec des frameworks comme Django et Flask.
- Science des données et Big Data : via des outils comme Pandas, NumPy et Matplotlib.
- Automatisation et scripting : pour automatiser des tâches répétitives.
- Développement de jeux : avec la bibliothèque Pygame.
- Cybersécurité : pour l'analyse et l'exploitation de données.

INTRODUCTION

Caractéristiques de Python

- Langage interprété : pas besoin de compilation.
- Facile à apprendre : syntaxe proche de l'anglais.
- Multi-plateforme : fonctionne sur Windows, Linux et macOS.
- Orienté objet et multiparadigme : supporte la programmation impérative, fonctionnelle et orientée objet.
- Communauté active : nombreux tutoriels, forums et bibliothèques.

VARIABLES

La création d'applications Web, de jeux et de moteurs de recherche implique de stocker et de travailler avec différents types de données. Ils le font en utilisant **des variables**.

- Une variable stocke une donnée et lui donne un nom spécifique. Par exemple:

```
spam = 5
```

La variable spam stocke maintenant le nombre 5.

Exercice :

Définissez la variable **my_variable** égale à la valeur 10. puis imprimer ce variable.

Écrivez votre code ci-dessous!

```
my_var = 10
```

```
print(" la valeur de mon variable =", my_var)
```

Exécuter

la valeur de mon variable = 10

VARIABLES

Les variables peuvent être de l'un des types suivants:

- Numérique
- Chaîne de caractère (Alphanumérique)
- Booléen
- Les variables numériques sont susceptibles de recevoir des nombres. Il existe également plusieurs types numériques tels que:

Type Numérique	Plage
Byte (octet)	0 à 255
Entier simple	-32 768 à 32 767
Entier long	-2 147 483 648 à 2 147 483 647
Réel simple	-3,40x10 ³⁸ à -1,40x10 ⁴⁵ pour les valeurs négatives 1,40x10 ⁻⁴⁵ à 3,40x10 ³⁸ pour les valeurs positives
Réel double	1,79x10 ³⁰⁸ à -4,94x10 ⁻³²⁴ pour les valeurs négatives 4,94x10 ⁻³²⁴ à 1,79x10 ³⁰⁸ pour les valeurs positives

```
# Définissez les variables aux valeurs listées dans les instructions!
```

```
my_int = 7
```

```
my_float = 1.23
```

```
print(" la valeur de mon variable", my_float)
```

Exécuter

Les types de données en python

Votre programme !

Modules Python

Tableaux :
Module `numpy`
(utilisation rare)

Classes et objets

Type de données de collection :

Séquences :

liste (`list`)

tuple (`tuple`)

ensemble (`set`)

ensemble fixe (`frozenset`)

Tableaux associatifs (`hash`) :

dictionnaire (`dict`)

dictionnaire ordonné

(`OrderedDict`)

Type de données de base :

Nombres :

booléen (`bool`)

entier (`int`)

flottant (`float`)

Textes :

chaîne de

caractères (`str`)

chaîne binaire (`bytes`)

Fonction

Fichier

(`file`)

VOUS AVEZ ÉTÉ AFFECTÉ !

Maintenant, vous savez comment utiliser les variables pour stocker des valeurs. Dites `my_int = 7`. Vous pouvez changer la valeur d'une variable en la "réaffectant", comme ceci:

```
my_int = 3
```

```
my_int = 7  
my_int = 3  
print(" la valeur de my_int", my_int)
```

Exécute

la valeur de my_int = 3

L'INSTRUCTION D'AFFECTATION

L'instruction d'affectation permet d'attribuer une valeur (non définitive) à une variable.
L'affectation s'effectue en utilisant le symbole (=).

```
spam = 5  
ping = 9
```

- On peut affecter à une variable le contenu d'une autre variable

```
spam = ping
```

- On peut incrémenter la valeur d'une même variable sans utiliser une deuxième variable

```
spam = spam + 1
```

Écrivez votre code ci-dessous!

```
A = 10  
B = A + 3  
A = 3  
print(" A = ", A)  
print(" B = ", B)
```

Exécuter

```
A = 3  
B = 13
```

L'INSTRUCTION D'AFFECTATION

Exercice :

écrire un algorithme permettant d'échanger les valeurs de deux variables A et B, et ce quel que soit leur contenu préalable.

Écrivez votre code ci-dessous!

```
A = 5
B = 6
C = A
A = B
B = C
print(" A = ", A)
print(" B = ", B)
print(" C = ", C)
```

Exécuter

```
A = 6
B = 5
C = 5
```

BOOLÉENS

Les nombres sont un type de données que nous utilisons dans la programmation. Un deuxième type de données s'appelle un **booléen**.

- Un booléen est comme un interrupteur d'éclairage. Il ne peut avoir que deux valeurs. Tout comme un interrupteur ne peut être activé ou désactivé, un booléen ne peut être que Vrai (True) ou Faux (False.).
- Vous pouvez utiliser des variables pour stocker des booléens comme ceci:

```
a = True  
b = False
```

Exercice : Définissez les variables suivantes sur les valeurs correspondantes:

- my_int à la valeur 7
- my_float à la valeur 1.23
- my_bool à la valeur True

```
# Définissez les variables aux valeurs listées  
dans les instructions!
```

```
my_int = 7  
my_float = 1.23  
my_bool = True  
print("la valeur de mon variable", my_bool)
```

Exécuter

la valeur de mon variable = True

COMMENTAIRES

Les commentaires rendent votre programme plus facile à comprendre. Lorsque vous regardez votre code ou que d'autres veulent collaborer avec vous, ils peuvent lire vos commentaires et facilement comprendre ce que fait votre code.

- Le signe # est pour les commentaires. Un commentaire est une ligne de texte que Python n'essaiera pas d'exécuter en tant que code. C'est juste pour les humains à lire.
- Vous pouvez écrire un commentaire multi-ligne, en commençant chaque ligne avec #, cela peut être une douleur.
- Au lieu de cela, pour les commentaires sur plusieurs lignes, vous pouvez inclure le bloc entier dans un ensemble de guillemets(" " " ")

```
# Écrivez votre commentaire ici  
# Écrivez un autre
```

```
"""
```

```
Siégeant de ta tasse jusqu'à ce que ça coule,  
Saint Graal.
```

```
"""
```

Exécute

LES MATHS

Vous pouvez ajouter, soustraire, multiplier, diviser des nombres comme celui-ci

```
addition = 72 + 23  
subtraction = 108 - 204  
multiplication = 108 * 0.5  
division = 108 / 9
```

Exercice : Définissez la variable **count** égale à la somme de deux grands nombres.

```
count = 10000 + 50000  
print(count)
```

Exécute

60000

LES MATHS: EXPONENTIATION

Tout ce calcul peut être fait sur une calculatrice, alors pourquoi utiliser Python?

- ✓ Parce que vous pouvez combiner les mathématiques avec d'autres types de données (par exemple, des booléens) et des commandes pour créer des programmes utiles. Les calculatrices se contentent de chiffres.

Maintenant, travaillons avec les exposants.

```
eight = 2**3
```

Exercice :

Créez une nouvelle variable appelée **eggs** et utilisez des exposants pour définir **eggs** égal à 100.

```
eggs = 10**2  
print(eggs)
```

Exécute

100

LES MATHS: MODULO

Notre opérateur final est modulo. Modulo renvoie le reste d'une division. Donc, si vous tapez $3 \% 2$, cela retournera 1, car 2 va en 3 une fois, avec 1 restant.

```
un = 3 % 2
```

Exercice :

Utilisez modulo pour définir un variable `spam` égal à 3.

```
spam = 7 % 4  
print(spam)
```

Exécute

3

Synthèse

```
# Écrivez votre code ci-dessous!  
A = 5 # Variable A  
B = A + 4 # Affectation '='  
A = A + 1  
my_int = 7 # Variable entier  
my_float = 1.23 # Variable réel  
my_bool = True # Variable Boolean (or False)  
count = 10000 + 50000 # addition  
eggs = 10**2 # exponentiel  
spam = 7 % 4 # Modulo  
print(" A = ", A) # Impression
```

SYNTHÈSE DE PYTHON

Jusqu' à présent, vous avez appris sur:

- Variables, qui stockent des valeurs pour une utilisation ultérieure
- Types de données, tels que les nombres et les booléens
- Commentaires, qui facilitent la lecture de votre code
- Opérations arithmétiques, y compris +, -, *, /, ** et %

STRING

Un autre type de données utile est la chaîne de caractère. Une chaîne peut contenir des lettres, des chiffres et des symboles.

```
name = "Ryan"  
age = "19"  
food = "cheese"
```

Les chaînes de caractères doivent être entre guillemets.

Exercice :

- Créez une nouvelle variable et attribuez-lui la chaîne "Hello life!".
- Définissez et imprimer les variables suivantes
 - Affecter César à "Graham"
 - Affecter praline à "John"
 - Affecter Viking à "Ragnar"

```
brian = "hello life!"
```

```
# Affectez vos variables ci-dessous, chacune  
sur sa propre ligne!
```

```
César = "Graham"
```

```
praline = "John"
```

```
Viking = "Ragnar"
```

```
# afficher vos variables
```

```
print(César)
```

```
print(praline)
```

```
print(Viking)
```

Exécute

Graham

John

Ragnar

STRING: ACCÈS PAR INDEX

Chaque caractère d'une chaîne est attribué un numéro. Ce nombre est appelé l'indice. Consultez le diagramme dans l'éditeur.

```
c = "chats"[0]  
n = "Ryan"[3]
```

Dans l'exemple ci-dessus, nous créons une nouvelle variable appelée c et le mettre à « c », le caractère à l'indice zéro de la chaîne « chats ».

En Python, nous commençons à compter l'indice de zéro au lieu d'un.

Exercice :

Quelle est le résultat de « print(fifth_letter)

»

"""

La chaîne "PYTHON" a six caractères, numérotés de 0 à 5, comme indiqué ci-dessous:

```
+---+---+---+---+---+---+  
| P | Y | T | H | O | N |  
+---+---+---+---+---+---+  
0  1  2  3  4  5
```

Donc, si vous vouliez "Y", vous pourriez simplement taper "PYTHON" [1] (toujours compter à partir de 0!)

"""

```
fifth_letter = "MONTY"[4]  
print(fifth_letter)
```

Exécute

Y

Chaînes de caractères

- Les chaînes de caractère s'écrivent entre guillemets (simples ou doubles)
 - > `Structure="Département d'Informatique TIC"`
 - > `chaine_vide= " "`
- Les chaînes de caractère sont en Unicode
 - Cela permet de gérer toutes les langues du monde et des symboles
- Pas de type caractère en Python
 - On utilise une chaîne d'un seul caractère
- Caractères spéciaux
 - Retour à la ligne : `"\n"`
 - Tabulation : `"\t"`
 - Antislash : `"\\"`

Chaînes de caractères

- Chaînes avec des guillemets à l'intérieur
 - « Antislasher » les guillemets à l'intérieur de la chaîne : `"Il se nomme \"Dahirou\"."`
 - Tripler les guillemets extérieurs : `"""Il se nomme "Dahirou".""""`
 - Alternier guillemets doubles et simples : `'Il se nomme "Dahirou".'`

Chaînes de caractères

■ Opérations sur les chaînes

- ◆ Obtenir la longueur d'une chaîne (= le nombre de caractères)
- ◆ Obtenir un caractère de la chaîne
- ◆ Obtenir une partie de la chaîne
- ◆ Concaténer deux chaînes
- ◆ Rechercher si une chaîne est incluse dans une autre

• Exemples avec `s = "Bonjour"`

`len(s)` $\Rightarrow 7$

`s[0]` $\Rightarrow$ "B" `s[-1]` $\Rightarrow$ "r"

`s[0:3]` $\Rightarrow$ "Bon"

`"Hi " + s`

`s.find("jour")` $\Rightarrow 3$ # Trouvé en position 3
(-1 si pas trouvé)

Chaînes de caractères

■ Opérations sur les chaînes •

- ◆ Passer une chaîne en minuscule

`s.lower()`

ou en majuscule

`s.upper()`

- ◆ Remplacer un morceau d'une chaîne

`s.replace("Bonjour", "Good morning")`

- ◆ Demander à l'utilisateur de saisir une chaîne

`saisie = input("Entrez un nom : ")`

Chaînes de caractères



Formatage de chaînes de caractère : inclure des variables dans une chaîne : %

```
>>> nom = "Gueye "
```

```
>>> prenom = "Dahirou"
```

```
>>> "Salut %s !" % prenom  
Salut Dahirou !
```

```
>>> "Bonjour %s %s !" % (prenom, nom)
```

```
>>> "Taux de réussite : %s %" % 100
```

```
Taux de réussite : 100 %
```



Attention aux faux nombres !

```
telephone1 = "00221 33 823 37 38"
```

```
telephone2 = 00221338233738
```

- Quel est le type de donnée des deux variables ci-dessus ?
- Pourquoi a-t-on choisi ce type de donnée pour le premier cas?

Conversions

- Il est souvent nécessaire de convertir un type de données vers un autre

- Les fonctions `int()`, `float()`, `bool()` et `str()` permettent de convertir une valeur vers un entier, un flottant et une chaîne de caractère

- `int("8")` $\Rightarrow$ 8
`str(8)` $\Rightarrow$ "8"

- `input()` retourne toujours une chaîne de caractères ; il faut penser à la transformer en entier ou en flottant si l'on demande la saisie d'un nombre !

`age = int(input("Entrez votre âge : "))`

`poids = float(input("Entrez votre poids : "))`

FONCTIONS INTÉGRÉES

Une fonction (ou function) est une suite d'instructions regrouper et définie par un nom et que on peut l'appeler avec ce nom.

Les fonctions intégrées au langage sont les fonctions prédéfinies dans un langage de programmation.

Les fonctions intégrées au langage sont relativement peu nombreuses : ce sont seulement celles qui sont susceptibles d'être utilisées très fréquemment.

Voyons comment nous pouvons changer les chaînes de caractères en utilisant ces méthodes.

MÉTHODES DE STRINGS

- Les méthodes de String permettent d'effectuer des tâches spécifiques sur les chaînes.
- Nous allons nous concentrer sur 4 méthodes:
 - `len()`
 - `lower()`
 - `upper()`
 - `str()`

Commençons par la plus simple `len (...)`, qui obtient la longueur (le nombre de caractères) d'une chaîne!

Exercice :

créer une variable nommée `parrot` et la définir sur la chaîne "Norwegian Blue" et afficher le nombre de caractères dans `parrot`.

```
# La sortie sera le nombre de lettres en  
"Norwegian Blue"!  
parrot = "Norwegian Blue"  
print(len(parrot))
```

Exécute

14

MÉTHODES DE STRINGS: LOWER() & UPPER()

Vous pouvez utiliser la méthode **lower ()** pour vous débarrasser de toutes les majuscules dans vos chaînes. Vous appelez **lower ()** comme ça:

```
"Ryan".lower()
```

Ce qui retournera « ryan ». Maintenant, votre chaîne est 100% minuscule!

Upper() est une méthode similaire pour faire une chaîne complètement en majuscule.

```
"Ryan".upper()
```

Ce qui retournera « RYAN ».

```
parrot = "Norwegian Blue"  
print(parrot.lower())  
print("Norwegian Blue".lower())  
print(parrot.upper())  
print("Norwegian Blue".upper())
```

Exécute

```
norwegian blue  
norwegian blue  
NORWEGIAN BLUE  
NORWEGIAN BLUE
```


MÉTHODES DE STRINGS: STR()

Maintenant, regardons `str ()`, ce qui est un peu moins simple. La méthode `str ()` transforme les non-chaînes en chaînes! Par exemple:

```
str(2)
```

Ce qui fait de 2 un " 2 " .

C'est quoi la différence.???????

```
pi = str(3.14) + 1  
print(pi)
```

Exécute

Traceback (most recent call last):

File "c:\Users\Asan\Desktop\UAM\Algo\test.py", line 2, in <module>

```
pi = str(3.14) + 1
```

```
~~~~~^~~~~
```

TypeError: can only concatenate str (not "int") to str

Imprimer des String

- La zone où nous avons écrit notre code s'appelle l'éditeur.
- La console (la fenêtre à droite de l'éditeur) est l'endroit où les résultats de votre code sont affichés.
- **print** affiche simplement votre code dans la console.

```
# instruction print
count = 1000
print("Imprimer dans le console")
print(count)
print("Imprimer un variable", count)
```

Exécuter

```
Imprimer dans le console
1000
Imprimer un variable 1000
```

Concaténation des String

- Vous connaissez les chaînes de caractères et vous connaissez les opérateurs arithmétiques. Maintenant, combinons les deux!

```
print("Life " + "of " + "Brian")
```

- Cela va imprimer la phrase Life of Brian

Concaténation des String

- Vous connaissez les chaînes de caractères et vous connaissez les opérateurs arithmétiques. Maintenant, combinons les deux!

```
print("Life " + "of " + "Brian")
```

- Cela va imprimer la phrase Life of Brian

L'opérateur « + » entre les chaînes les "ajoute" l'une après l'autre. Notez qu'il y a des espaces à l'intérieur des guillemets après la vie et de sorte que nous pouvons faire ressembler la chaîne combinée à 3 mots.

Imprimez la concaténation de "Spam and eggs" sur la ligne 3!

```
print("Spam" + " and " + "eggs")
```

Exécuter

Spam and eggs

LES MATHS

Python peut faire tout ce qui est sur une calculatrice, est plus?

- Avant d'essayer d'aller plus loin, voyons ce que Python sait déjà des racines carrées. Demandant à Python de faire la racine carrée de 25.

```
print(sqrt(25))
```

Maintenant, travaillons avec les exposants.

```
Traceback (most recent call last): File  
"python", line 2, in <module> NameError:  
name 'sqrt' is not defined
```

Avez-vous vu ça? Python a déclaré:

NameError: le nom '**sqrt**' n'est pas défini. Python ne sait pas encore ce que sont les racines carrées.

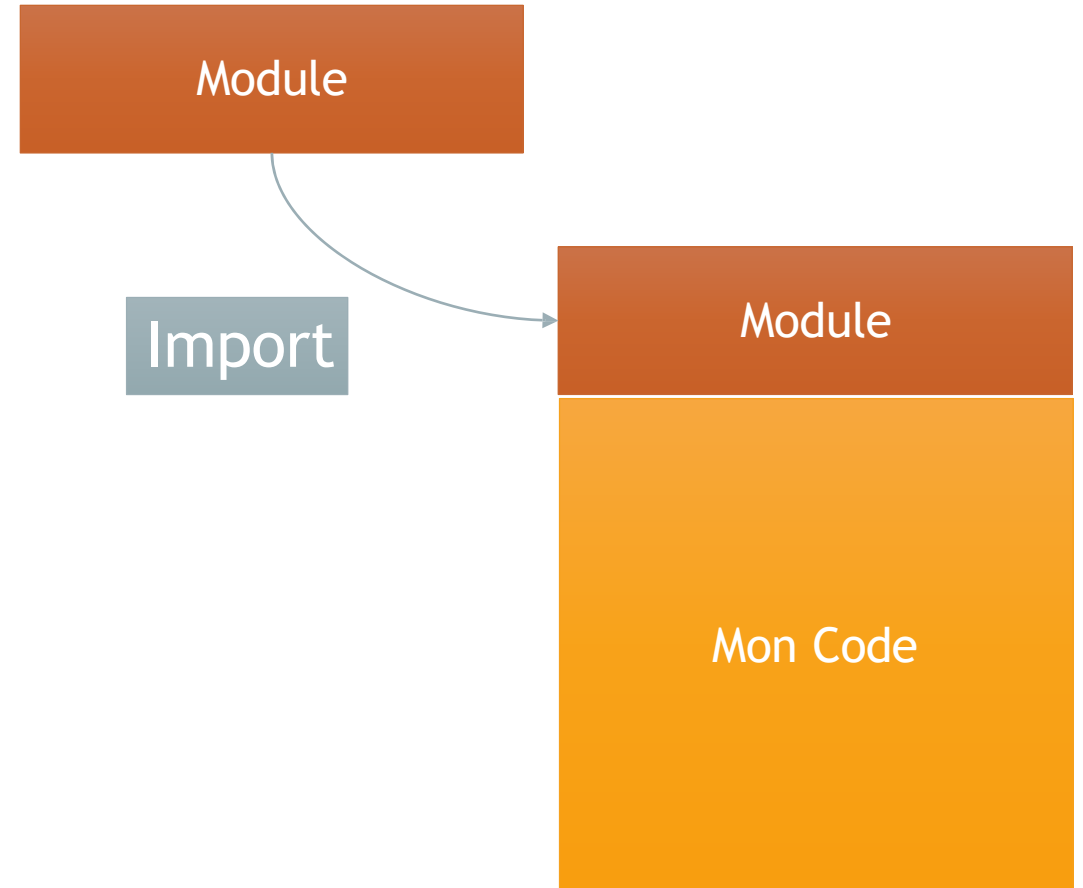
IMPORTATIONS

```
len("AAACGTR")
```

Les **fonctions intégrées** au langage sont relativement peu nombreuses : ce sont seulement celles qui sont susceptibles d'être utilisées très fréquemment. Les autres sont regroupées dans des fichiers séparés que l'on appelle des **modules**.

Les modules sont donc des fichiers qui regroupent un ensemble de fonctions. Il existe un grand nombre de modules pré-programmés qui sont fournis d'office avec Python.

Pour utiliser des fonctions de modules dans un programme, il faut au début du fichier **importer** ceux-ci.

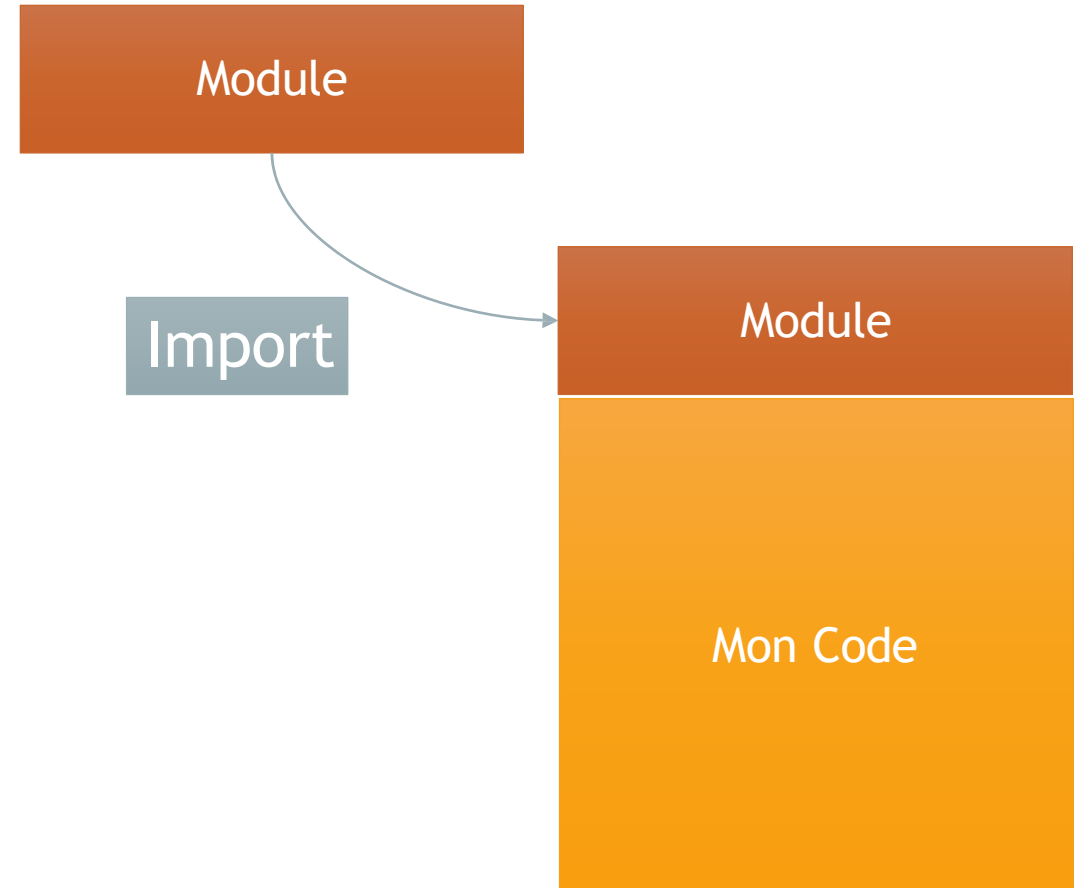


IMPORTATIONS: IMPORTATION GÉNÉRIQUE

Il existe un module Python nommé **math** qui inclut un certain nombre de variables et de fonctions utiles, et **sqrt ()** est l'une de ces fonctions. Pour accéder aux mathématiques, tout ce dont vous avez besoin est le mot-clé **import**. Lorsque vous importez simplement un module de cette façon, cela s'appelle une **importation générique**.

```
import math  
print(math.sqrt(25))
```

Pour utiliser des fonctions de modules dans un programme, il faut au début du fichier **importer** ceux-ci.



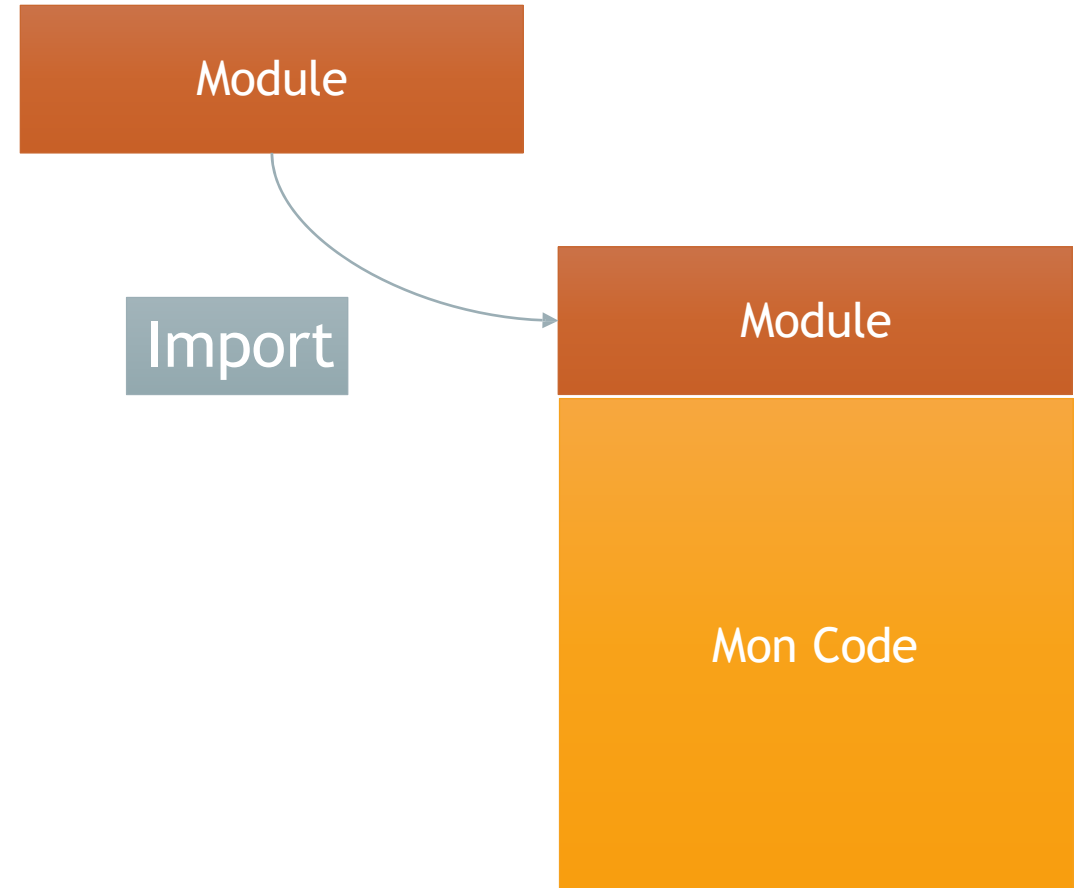
IMPORTATIONS DE FONCTIONS

Il est possible d'importer uniquement certaines variables ou fonctions d'un module donné. Extraire une seule fonction d'un module s'appelle une importation de fonction, et c'est fait avec le mot-clé **from**:

```
from math import sqrt  
print(sqrt(25))
```

Maintenant, vous pouvez simplement taper `sqrt ()` pour obtenir la racine carrée d'un nombre - plus `math.sqrt ()`!

Pour utiliser des fonctions de modules dans un programme, il faut au début du fichier **importer** ceux-ci.

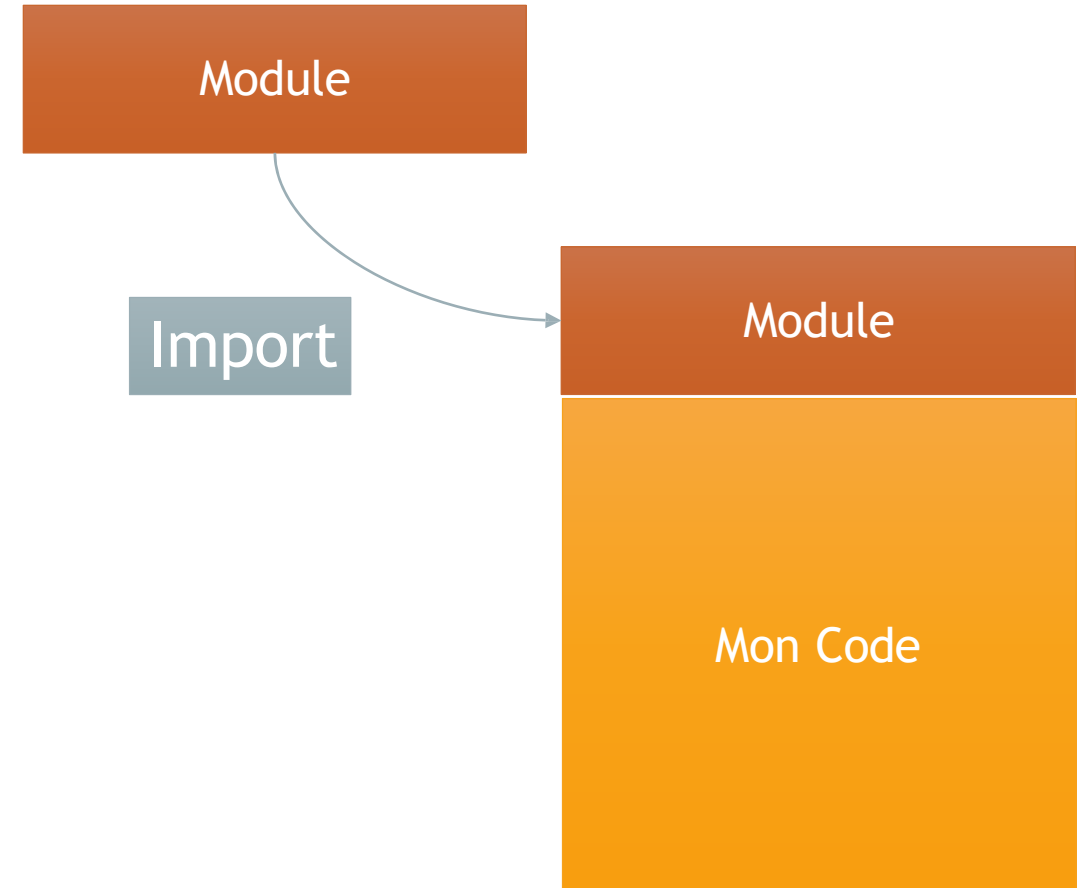


IMPORTATIONS UNIVERSELLES

Que faire si nous voulons toujours toutes les variables et les fonctions dans un module, mais ne veulent pas avoir à taper constamment des maths.?

```
from math import *  
print(sqrt(25))
```

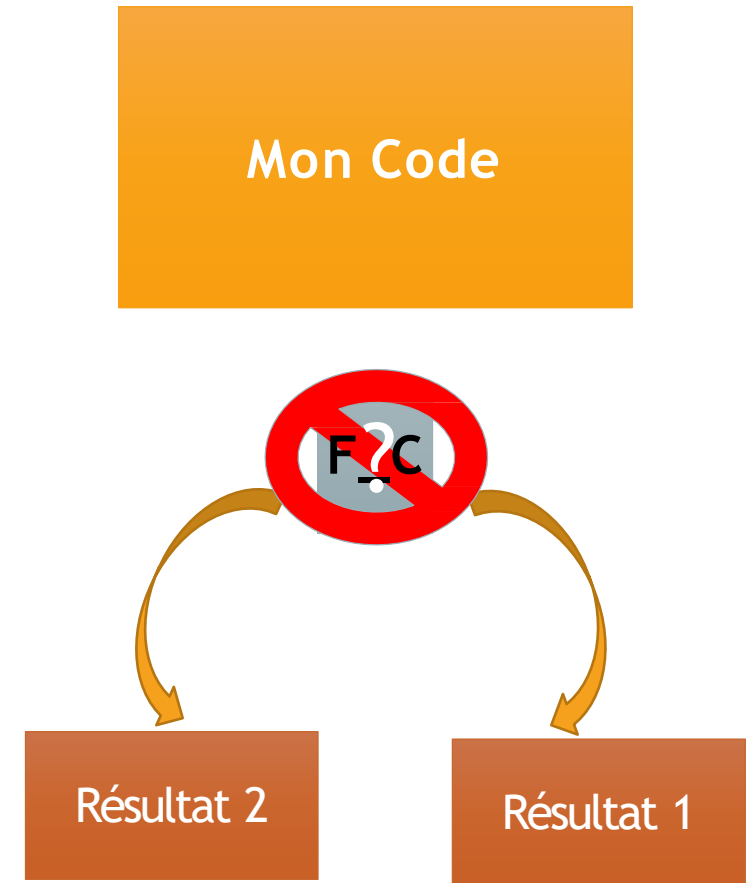
Pour utiliser des fonctions de modules dans un programme, il faut au début du fichier **importer** ceux-ci.



ALLER AVEC LE FLUX

Les programmes Python que nous avons écrits jusqu'ici ont eu un esprit unique: ils peuvent ajouter deux nombres ou imprimer quelque chose, mais ils n'ont pas la possibilité de choisir l'un de ces résultats par rapport à l'autre.

Le flux de contrôle nous donne cette possibilité de choisir parmi les résultats en fonction de ce qui se passe d'autre dans le programme.



Comparer !

- Commençons par l'aspect le plus simple du flux de contrôle: **les comparateurs**. Il y a six:

Égal à (==)

Different de (!=)

Inférieur à (<)

Inférieur ou égal à (<=)

Supérieur à (>)

Supérieur ou égal à (>=)

Notez que == compare si deux choses sont égales, et = assigne une valeur à une variable.

1. 3 == 3

True

2. 3 == 6

False

1. 3 != 6

True

2. 3 != 3

False

1. 3 < 6

True

2. 6 < 3

False

Comparer !

- Exercice
- Définissez chaque variable sur Vrai ou Faux en fonction de ce que vous pensez que le résultat sera. Par exemple, $1 < 2$ sera True, car 1 est inférieur à 2.

```
# Réglez ce paramètre sur Vrai si  $17 < 328$  ou  
False si elle est pas.
```

```
ResBool =
```

```
# Réglez ce paramètre sur Vrai si  $100 == (2$   
* 50) ou False autrement.
```

```
ResBool =
```

```
# Réglez ce paramètre sur Vrai si  $-22 > -18$   
ou False si elle est pas.
```

```
ResBool =
```

```
# C'est quoi le resultat de print ResBool
```

```
ResBool =  $(-22 > -18)$ 
```

```
print(ResBool)
```

Utiliser les Booléens

- Les **opérateurs booléens** comparent les instructions et produisent des valeurs booléennes. Il y a trois opérateurs booléens:

and

qui vérifie si les deux instructions sont vraies;

or

qui vérifie si au moins l'une des déclarations est True;

not

ce qui donne le contraire de la déclaration.

Exécuter

Les Opérateurs Booléens

- L'opérateur booléen **and** renvoie **True** lorsque les expressions des deux côtés de **and** sont vraies.

2 < 3 and 3 < 4

True

2 > 3 and 3 < 4

False

X	Y	X and Y
True	True	True
False	True	False
True	False	False
False	False	False

Les Opérateurs Booléens

```
A = False and False
B = -(-(-(-2))) == -2 and 4 >= 16**0.5
C = 19 % 4 != 300 / 10 / 10 and False
D = -(1**2) < 2**0 and 10 % 10 <= 20 - 10*2
print(A, B, C, D)
```

and

qui vérifie si les deux instructions sont vraies;

- L'opérateur booléen **and** renvoie **True** lorsque les expressions des deux côtés de **and** sont vraies.

2 < 3 and 3 < 4

True

2 > 3 and 3 < 4

False

Exécuter

False False False True

Les Opérateurs Booléens

- L'opérateur booléen **or** retourne **True** lorsque au moins une expression de chaque côté de **ou** est vraie. Par exemple:

2 < 3 or 3 < 4

True

2 > 3 or 3 < 4

True

2 > 3 or 3 > 4

False

or

qui vérifie si au moins l'une des déclarations est True;

X	Y	X or Y
True	True	True
False	True	True
True	False	True
False	False	False

Les Opérateurs Booléens

or

qui vérifie si au moins l'une des déclarations est True;

- L'opérateur booléen **or** retourne **True** lorsque au moins une expression de chaque côté de **ou** est vraie. Par exemple:

2 < 3 or 3 < 4

True

2 > 3 or 3 < 4

True

2 > 3 or 3 > 4

False

```
A = 2**3 == 108 % 100 or 'Rymond' == 'Redington'
B = True or False
C = 100**0.5 >= 50 or False
D = 1**100 == 100**1 or 3 * 2 * 1 != 3 + 2 + 1
print(A, B, C, D)
```

Exécuter



Les Opérateurs Booléens

not

ce qui donne le contraire de la déclaration.

- L'opérateur booléen **not** renvoie **True** pour les fausses instructions et **False** pour les vraies instructions. Par exemple:

not False

True

not 2 > 3

True

not 3 < 4

False

not

X	Not X
True	False
False	True

Les Opérateurs Booléens

not

ce qui donne le contraire de la déclaration.

- L'opérateur booléen **not** renvoie **True** pour les fausses instructions et **False** pour les vraies instructions. Par exemple:

not False

True

not 2 > 3

True

not 3 < 4

False

```
A = not True
B = not 3**4 < 4**3
C = not 10 % 3 <= 10 % 2
D = not 3**2 + 4**2 != 5**2
E = not not False
print(A,B,C,D,E)
```

Exécuter



Utiliser les Boolean

```
bool= (2 <= 2) and "Alpha" == "Bravo"  
print(bool)
```

Exécuter

False

Syntaxe de l'instruction conditionnelle

The big IF

- **if** est une instruction conditionnelle qui exécute du code spécifié après avoir vérifié si son expression est True.
- Voici un exemple de syntaxe d'instruction if:

```
if 8 < 9:  
    print("Eight is less than nine!")
```

Dans cet exemple, `8 < 9` est l'expression conditionnelle à contrôler et l'impression "Huit est inférieur à neuf!" est le code spécifié.

```
reponse = "Y"  
answer = "Left"  
if answer == "Left":  
    print("This is the Verbal Abuse Room")
```

Exécuter

```
1. This is the Verbal Abuse Room,  
   you heap of parrot droppings!
```

Syntaxe de l'instruction conditionnelle

The big IF

Exercice 1:

Ecrire un algorithme qui demande un nombre à l'utilisateur, et l'informe ensuite si ce nombre est positif ou négatif (le cas où le nombre est nul n'est pas traité).

```
Enter = input ()
```

```
Enter = input("message du commande ")
```

```
reponse = input("Entrer un numero!")  
if reponse > 0:  
    print("ton numero est positif")
```

Exécuter

1. Entrer un numero!
2. --

Syntaxe de l'instruction conditionnelle

The big IF

- **if** l'expression conditionnelle :
 - Code spécifié.
- l'expression conditionnelle =
 - $(2 \leq 2)$ and "Alpha" == "Bravo "
 - $\text{Var1} > \text{Var2}$
 - $\text{Bool1} == \text{True}$
 - $\text{Trouvé} \neq \text{False}$

```
reponse = "Y"  
answer = "Left"  
if answer == "Left " and reponse != "y":  
    print("This is the Verbal Abuse Room!")
```

Exécuter

1. This is the Verbal Abuse Room!

Syntaxe de l'instruction conditionnelle

The big IF

Exercice 2 :

Ecrire un programme demandant à l'utilisateur de donner sa moyenne au bac et affichant s'il est admis,

```
note = input("Entrez votre moyenne obtenue au  
bac ")  
note = float(note) # convertir la note en réel car la  
fonction input récupère en chaîne de caractères  
if note >= 10 :  
    print("Vous êtes admis")
```

Exécuter

1. Entrer quelque chose!

Syntaxe de l'instruction conditionnelle

The big IF / ELSE

L'instruction **else** complète l'instruction **if**. Une paire **if / else** dit:

Si cette expression est vraie,

exécutez ce bloc de code en retrait,

sinon,

exécutez ce code après l'instruction **else**."

Contrairement à si, sinon ne dépend pas d'une expression.

```
if 8 > 9:  
    print("I don't printed!")  
else:  
    print("I get printed!")
```

Exécuter

1. I get printed!

Syntaxe de l'instruction conditionnelle

Exercice 2 :

Ecrire un programme demandant à l'utilisateur de donner sa moyenne au bac et affichant s'il est admis, ainsi que sa mention.

[12-14] mention AB

[14-16] mention B

[16 - >16] mention A

```
note = input ("Entrez votre moyenne obtenue au bac")
note = float(note)
if note >= 12 and note <14 :
    print("Mention AB")
if note >= 14 and note <16 :
    print("Mention B")
if note >=16 :
    print("Mention TB")
```

Exécuter



Syntaxe de l'instruction conditionnelle

Exercice 2 :

Ecrire un programme demandant à l'utilisateur de donner sa moyenne au bac et affichant s'il est admis, ainsi que sa mention.

[12-14] mention AB

[14-16] mention B

[16 - >16] mention A

```
note = input("Entrez votre moyenne obtenu aubac ")
note = float(note)
if note >= 10 :
    print ("Vous êtes admis")
if note >= 12 and note <14 :
    print ("Mention AB")
if note >= 14 and note<16 :
    print ("Mention B")
if note >=16 :
    print ("Mention TB")
else:
    print ("Vous n'êtes pas admis")
```

Syntaxe de l'instruction conditionnelle

The big elif

elif est l'abréviation de "else if". Cela signifie exactement ce que cela ressemble: "sinon, si l'expression suivante est vraie, faites ceci!"

```
if 8 > 9:  
    print("I don't get printed!")  
elif 8 < 9:  
    print("I get printed!")  
else:  
    print("I also don't get printed!")
```

Dans l'exemple ci-dessus, l'instruction elif n'est vérifiée que si l'instruction if d'origine est False.

```
note = input("Entrez votre moyenne obtenue au bac ")  
note = float(note)  
if note > 10 :  
    print("Vous êtes admis")  
elif note < 10 :  
    print("Vous êtes Fichu ")  
elif note = 10 :  
    print("Vous êtes sauver")  
else :  
    print(" y a un problème la ")
```

Exécuter



PYTHON



Introduction aux listes

- Les listes sont un type de données que vous pouvez utiliser pour stocker une collection de différentes informations en tant que séquence sous un même nom de variable. (Les types de données dont vous avez déjà entendu parler incluent les chaînes, les nombres et les booléens.)
- Vous pouvez attribuer des éléments à une liste avec une expression de la forme

```
list_name = [item_1, item_2]
```

- avec les éléments entre parenthèses. Une liste peut également être vide:
`empty_list = []`.

```
zoo_animals = ["Tigre", "lion", "Singe" ]  
if len(zoo_animals) == 3:  
    print(" le 1ere animal dans le zoo: " + zoo_animals[0])  
    print(" le 2eme animal dans le zoo: " + zoo_animals[1])  
    print(" le 3eme animal dans le zoo: " +  
zoo_animals[2])
```

Exécuter

- On doit ajouter un élément dans la liste sinon ce code n'affiche rien.

Listes: Accès par index

- Vous pouvez accéder à un élément individuel de la liste par son index.
- Rappel : Un index est comme une adresse qui identifie la place de l'élément dans la liste.
- The index appears directly after the list name, in between brackets, like this:

```
list_name[index]
```

```
numbers = [5, 6, 7, 8]  
print(numbers [0] + numbers [2])
```

Exécuter

Exemple : Ecrire une déclaration qui imprime le résultat de l'ajout du deuxième et quatrième élément de la liste. Assurez-vous d'accéder à la liste par index!

Listes: Longueur de la liste

- Une liste n'a pas une longueur fixe. Vous pouvez ajouter des éléments à la fin d'une liste quand vous le souhaitez!

```
letters = ['a', 'b', 'c']  
letters.append('d')  
print(len(letters))  
print(letters)
```

Exécuter

4

['a', 'b', 'c', 'd']

Listes: Découpage des listes

L'opérateur d'indexage:

- L'opérateur d'indexage ([]) permet aussi de sélectionner des sous-chaines selon leurs indices.
- On appelle cette technique le slicing (« découpage en tranches »).

- `chain = "123456789"`
- `print( chain[1:3] )`
- `>>> 23`
- `print( chain[1:len(chain)] )`
- `>>> 23456789`

Listes: Découpage des listes

Parfois, vous voulez seulement accéder à une partie d'une liste.

- D'abord, nous créons une liste appelée lettres.
- Ensuite, nous prenons une sous-section de la liste et la stockons dans slice . Nous faisons cela en définissant les indices que nous voulons inclure après le nom de la liste: letters [1: 3].
- En Python, lorsque nous spécifions une partie d'une liste de cette manière, nous incluons l'élément avec le premier index, 1, mais nous excluons l'élément avec le second index, 3.
- Ensuite, nous imprimons la découpe, qui imprimera ['b', 'c']. Rappelez-vous, dans Python, les indices commencent toujours à 0, donc l'élément 1 est en fait b.

```
letters = ['a', 'b', 'c', 'd', 'e']  
slice = letters[1:3]  
print(slice)  
print(letters)
```

Exécuter

```
['b', 'c']  
['a', 'b', 'c', 'd', 'e']
```

Listes: Découpage des listes et de Chaîne de caractères

- En fait, vous pouvez considérer les chaînes comme des listes de caractères: chaque caractère est un élément séquentiel dans la liste, à partir de l'index 0.
- Vous pouvez trancher une chaîne exactement comme une liste!

```
• Chaine = "Juliette"
• print ch[:3]    # les 3 premiers caractères
• >>>Jul
• -----
• -
• print(ch[3:])   # tout sauf les 3 premiers
caractères
• >>>lette
• Chaine = "Juliette"
• -----
• print(ch[:-3])  # tout sauf les 3 derniers
caractères
• >>>Julle
• -----
• print(ch[-3:])  # les 3 dernies caractères
• >>>>>Tte
```

Listes: quelques opérations

Parfois, vous voulez seulement accéder à une partie d'une liste.

- Nous pouvons insérer des éléments dans une liste.

```
animals.insert(1, "dog")
```

- Parfois, vous devez rechercher un élément dans une liste.

```
animals.index("bat")
```

```
print(animals.index("bat"))
```

Exécuter

Listes: quelques opérations

- Si votre liste est un désordre brouillé, vous aurez peut être besoin de la trier.

```
if "Y" in chromosomes:  
    print("C'est un garçon !")
```

- Attention, «=» ne copie pas les listes !

```
List1 = List2
```

- Pour copier une liste :

```
List3 =  
List1[:]
```

```
1. L1 = [5, 3, 1, 2, 4]
```

```
2. L2 = []
```

```
1. L2=L1
```

```
2. L3=L1[:]
```

```
3. L1.remove(5)
```

```
4. print L1
```

```
5. print L2
```

```
6. print L3
```

```
[3, 1, 2, 4]  
[3, 1, 2, 4]  
[5, 3, 1, 2, 4]
```

Listes: quelques opérations

- Si votre liste est un désordre brouillé, vous aurez peut être besoin de la trier.

```
animals = ["cat", "ant", "bat"]  
animals.sort()
```

- D'abord, nous créons une liste appelée animaux avec trois chaînes. Les chaînes ne sont pas dans l'ordre alphabétique.
- Ensuite, nous trions les animaux dans l'ordre alphabétique. Notez que `.sort ()` modifie la liste plutôt que de retourner une nouvelle liste.
- Pour supprimer un élément de la liste utiliser :

```
animals.remove("cat")
```

Exécuter

Listes: Pour un et tous

Si vous voulez faire quelque chose avec chaque élément de la liste, vous pouvez utiliser une boucle **'for'**.

- Nous pouvons insérer des éléments dans une liste.

```
for variable in list_name:  
    # Faire des choses!
```

- Pour le nom de variable qui suit le mot-clé **'for'**; il sera attribué la valeur de chaque élément de la liste à son tour.
- Ensuite, dans list_name, le nom de la liste sur laquelle la boucle fonctionnera. La ligne se termine par un deux-points (:) et le code en retrait qui le suit sera exécuté une fois par élément dans la liste.

```
for number in my_list:
```

Solution

```
my_list = [1,9,3,8,5,7]  
for number in my_list :  
    print(2 * number)
```

Listes: Pour un et tous

- Si vous voulez obtenir une liste de nombre utiliser la boucle range

```
range([debut], fin, [pas])
```

- Remarque bien :

```
print range(10)
> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print range(4, 10)
> [4, 5, 6, 7, 8, 9]
print range(4, 10, 2)
> [4, 6, 8]
```

```
for i in range(10) :
    print(i)
```

Solution

```
0
1
2
...
9
```


Listes: Pour un et tous

- Range peut être utilisé pour boucler sur les indices, et non pas sur les éléments d'une liste

```
adn = "atcacgtta"  
for i in range(len(adn)) :  
    print("la base n", i, "est", adn[i])
```

Solution

```
la base n° 0 est a  
la base n° 1 est t  
la base n° 2 est c  
...  
la base n° 8 est a
```

LISTES MULTIPLE

- Il est également courant de devoir parcourir deux listes à la fois. C'est là que la fonction zip intégrée est très utile.
- zip créera des paires d'éléments lorsque deux listes seront passées et s'arrêtera à la fin de la liste la plus courte.
- zip peut gérer trois listes ou plus!

```
list_a = ['V', 9, 17, 15, 19]  
list_b = ['A', 4, 8, 10, 30, 40, 50, 80, 90]
```

```
for a, b in zip(list_a, list_b):  
    # Add your code here!  
    print(a,b)
```

```
1.  V A  
2.  9 4  
3.  17 8  
4.  15 10  
5.  19 30
```

LISTES MULTIPLE

- Il est également courant de devoir parcourir deux listes à la fois. C'est là que la fonction zip intégrée est très utile.
- zip créera des paires d'éléments lorsque deux listes seront passées et s'arrêtera à la fin de la liste la plus courte.
- zip peut gérer trois listes ou plus!

```
list_a = ['V', 9, 17, 15, 19]
list_b = ['A', 4, 8, 10, 30, 40, 50, 80, 90]
list_c = ['C', 9, 17, 15, 19]
```

```
for a, b, c in zip(list_a, list_b, list_c):
    # Add your code here!
    print(a,b,c)
```

```
1.  V A C
2.  9 4 9
3.  17 8 17
4.  15 10 15
5.  19 30 19
```

Boucle: Pendant que tu es là

While

- La boucle **while** est similaire à une instruction if: elle exécute le code à l'intérieur si une condition est vraie.
- La différence est que la boucle **while** continuera à exécuter tant que la condition est vraie.
- En d'autres termes, au lieu d'exécuter si quelque chose est vrai, il s'exécute pendant que cette chose est vraie.

```
count = 0
if count < 5:
    print("Hello, je suis un if et count = ",
count)
```

```
while count < 5:
    print("Hello, count = ", count)
    count = count + 1
```

Solution

Boucle: Pendant que tu es là

While

- **La condition** est l'expression qui décide si la boucle va continuer à être exécutée ou non.
- Le variable condition contient la valeur True
- La boucle while vérifie si la condition a la valeur True. C'est le cas, donc la boucle est entrée.
- L'instruction d'impression est exécutée.
- Le variable condition est définie sur False.
- La boucle while vérifie à nouveau si loop_condition a la valeur True. Ce n'est pas le cas, donc la boucle n'est pas exécutée une seconde fois.
- À l'intérieur d'une boucle while, vous pouvez faire tout ce que vous pourriez faire ailleurs, y compris les opérations arithmétiques

```
while loop_condition:  
    print("Je suis une boucle")  
    loop_condition = False
```

Solution

Je suis une boucle

Boucle: Pendant que tu es là

While

- A faire;

Créer une boucle **while** qui imprime tous les nombres de 1 à 10 au carré (1, 4, 9, 16, ..., 100), chacun sur leur propre ligne.

```
num = 1
while num < 11 :
    print(num**2)
    num=num+1
```

Solution

```
1
4
9
16
25
..
81
100
```

Boucle: Pendant que tu es là

While

- Une application courante d'une boucle **while** consiste à vérifier l'entrée de l'utilisateur pour voir si elle est valide.
- Par exemple, si vous demandez à l'utilisateur d'entrer o ou n et qu'ils saisissent à la place 7, vous devez les inviter à entrer une nouvelle fois.

```
choix = input("aimez-vous le cours? (o/n)")
```

```
while choix != 'o' and choix != 'n' :
```

```
    choix = input("pardon j'ai pas compris essayer encore: ")
```

Solution

Boucles Infinies

While

- Une boucle infinie est une boucle qui ne sort jamais. Cela peut arriver pour plusieurs raisons:
 - La condition de boucle ne peut pas être fausse (par exemple pendant que $1! = 2$)
 - La logique de la boucle empêche la condition de boucle de devenir fausse.

```
while count > 0:  
    count += 1 # a la place de count -= 1
```

Solution

BREAK=SE BRISER.

While

- **break** est une instruction d'une ligne qui signifie "quitter la boucle actuelle".
- Une autre façon de faire sortir notre boucle de comptage et arrêter l'exécution est avec l'instruction **break**.

```
while count > 0:  
    count += 1 # a la place de count -= 1
```

```
from random import randint  
  
random_number = randint(1, 10)  
guesses_left = 3  
  
while guesses_left > 0:  
    guess = int(raw_input("Your guess: "))  
    if num == random_number:  
        print("You win!")  
        break  
    guesses_left -= 1  
else:  
    print("you lose!")
```

- Dictionnaires, Tuples et ensembles

Dictionnaires

- Les types de données composites que nous avons abordés jusqu'à présent (chaines, listes) étaient tous des séquences, c'est à dire des suites ordonnées d'éléments.
- Dans une séquence, il est facile d'accéder à un élément quelconque à l'aide d'un index (un nombre entier), mais à la condition expresse de connaître son emplacement.
- Les *dictionnaires* que nous découvrons ici constituent un autre *type composite*.
- Ils ressemblent aux listes dans une certaine mesure (ils sont modifiables comme elles), mais ne sont pas des séquences.
- En revanche, nous pourrions accéder à n'importe lequel d'entre eux à l'aide d'un index spécifique que l'on appellera une *clé*, laquelle pourra être alphabétique, numérique, ou même d'un type composite sous certaines conditions.

Dictionnaires

- Un tableau associatif qui fait correspondre des valeurs à des clefs : **dict**

```
dict = { clef1 : valeur1, clef2 : valeur2, ... }
```

```
>>> définitions = {"os" : "système d'exploitation", "python" : "langage de  
programmation", "dictionnaire" : "association clef-valeur"}
```

```
>>> fiche = {"nom": "Gueye", "prenom": "Dahirou"}
```

```
>>> materiel = {}
```

```
>>> materiel['computer'] = 'ordinateur'
```

```
>>> materiel['mouse'] = 'souris'
```

```
>>> materiel['keyboard'] = 'clavier'
```

```
>>> print(materiel)
```

```
{'computer': 'ordinateur', 'keyboard': 'clavier', 'mouse': 'souris'}
```

Opérations sur les dictionnaires

- La recherche est optimisée dans les dictionnaires (table de hachage)
- Principales opérations :
 - Nombre de clés ou de valeur : `len (definitions)` -> 3
 - Obtenir la valeur associée à une clé: `definitions ["python"]` -> langage de programmation
 - Ajouter un couple de clé: valeur: `definitions["flottant"] = "nombre décimal"`
 - supprimer une clé et la valeur associée: `del definitions["os"]`
 - Test d'appartenance

```
>>> if "os" in definitions:  
...     print("Intègre un système")  
... else:  
...     print("Pas de système. Sorry") ...  
Pas de systèmes. Sorry
```

Opérations sur les dictionnaires



Récupérer une valeur dans un dictionnaire: get

```
>>> data = {"os": "systeme", "age": 30}
```

```
>>> data.get("os")
```

```
'système'
```

```
>>> data.get("adresse", "Adresse inconnue")
```

```
'Adresse inconnue'
```



Méthode keys()

- >>> print(data.keys())



Vérifier la présence d'une clé : haskey

- >>> data.has_key("os")
True



Méthode values()

- >>> print(data.values())

Dictionnaires et boucles

- Une boucle sur un dictionnaire parcourt les clefs :

```
for clef in définitions:  
    print(clef, " : ", définitions[clef])
```

- Récupérer les clés

- ```
for clef in définitions.keys(): print (clef)
```

- Récupérer les valeurs

```
for valeur in définitions.values() :
 print(valeur)
```

- Récupérer les couples (clés:valeurs)

```
for clef, valeur in définitions.items() :
 print(clef, " : ", valeur)
```

- Dictionnaire procédural :

```
quadruple = { i : 4 * i for i in range(10) }
```

## BOUCLEZ UN DICTIONNAIRE

- Une faiblesse de l'utilisation des itération (for, while) est que vous ne connaissez pas l'indice de ce que vous regardez. Généralement, ce n'est pas un problème, mais il est parfois utile de savoir à quel point de la liste vous êtes.
- Heureusement, la fonction **enumerate** intégrée facilite cette tâche.

```
choices = ['pizza', 'pasta', 'salad', 'nachos']

print('Your choices are:')
for index, item in enumerate(choices):
 print(index, item)
```

```
1. Your choices are:
2. 0 pizza
3. 1 pasta
4. 2 salad
5. 3 nachos
```



# Contrôle du flux d'exécution à l'aide d'un dico

```
dico = {'fer':fonctionA,
 'bois':fonctionC,
 'cuivre':fonctionB,
 'pierre':fonctionD,
 ... etc ...}
```

- Les deux se résument:

```
dico[materiau]
```

- On peut améliorer la technique avec `get()`

```
dico.get(materiau, fonctAutre)()
```

